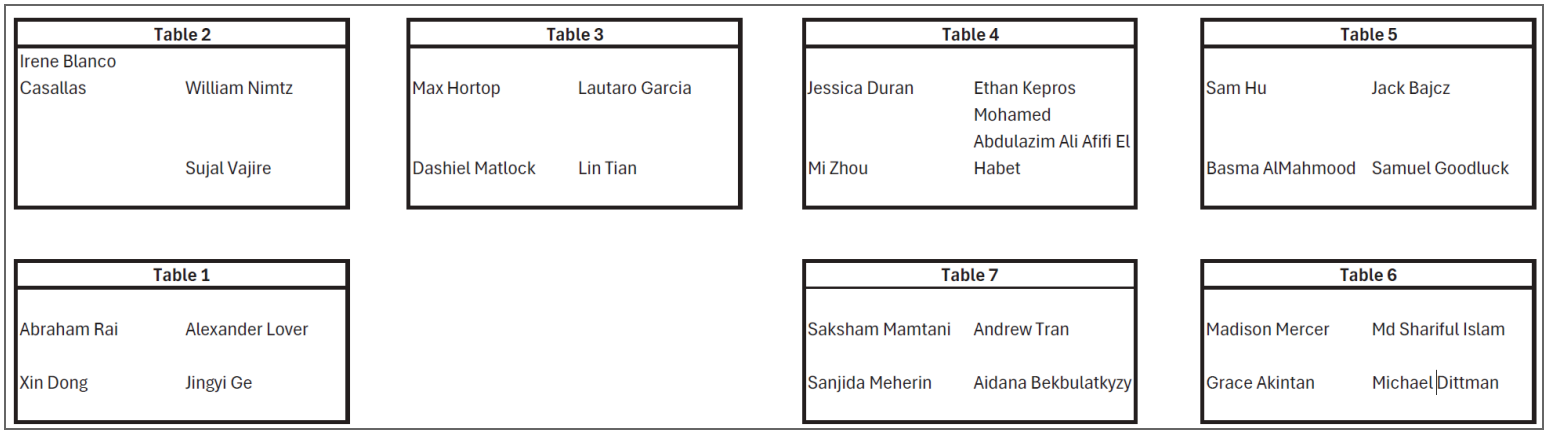


Table 2	
Irene Blanco Casallas	William Nimtz
	Sujal Vajire

Table 3	
Max Hortop	Lautaro Garcia
Dashiel Matlock	Lin Tian

Table 4	
Jessica Duran	Ethan Kepros
Mi Zhou	Mohamed Abdulazim Ali Afifi El Habet

Table 5	
Sam Hu	Jack Bajcz
Basma AlMahmood	Samuel Goodluck

Table 1	
Abraham Rai	Alexander Lover
Xin Dong	Jingyi Ge

Table 7	
Saksham Mamtani	Andrew Tran
Sanjida Meherin	Aidana Bekbulatkyzy

Table 6	
Madison Mercer	Md Shariful Islam
Grace Akintan	Michael Dittman

CMSE 801 - Day 8

Modeling with ordinary differential equations

Feb 5, 2026

General Course Announcements

- **Reminder**: Homework 1 is due this Feb 21.
    - *Don't wait until the last minute!* -- It shouldn't be overly challenging but it does take time to work through all of the components, which are designed to make sure you review *all* of the fundamentals we've covered up to this point.
- Also, we will have **our first quiz in class a week from today, Thursday, Feb 12**.
    - This will cover concepts that are covered by Homework 1 (so make sure you're feeling good about your Python fundamentals).
- Homework 2 should get released this next week. It will be due Saturaday, **Feb 28**.

- This will cover using Matplotlib and NumPy as well as a little bit of ODE content (which we're covering this week).

Questions/comments from Day 8 PCA

- Understanding and playing around with update equations
- Needed to dust off some old math knowledge!
- Some lack of clarity around ordinary different equations (ODEs) and how they are being used to model something and what they could be used for outside of the example provided.
- How is the "derivatives" function connected to the main code that will evolve the model?
    - This is the piece that the pseudocode question was trying to get at -- **we'll talk this through**.

ODEs can be used to model any evolving system that is evolving, if we know the *rates of change*, such as:

- How drugs are processed by the human body
- How physical systems move
- How disease spreads in communities
- How misinformation spreads via social media
- How pollutants propagate through water systems
- How populations evolve in various ecosystems, **and many more...**

To use these sorts of models, we need to know two things:

1. What the state of the system is ***at the beginning***
2. What the equations for the ***rates of change*** are, i.e. how does the system change? (this is where the derivatives come in!)

For our simple population model

Say we know that the population, P, at $t_0 = 0$ is 1 billion (i.e. $P(t_0) = 1 \text{ billion}$)
And we know that the *rate of change* for the population is given by:

$$\frac{dP}{dt} = kP\left(1 - \frac{P}{C}\right)$$

Then we can find a population at some new time, $t_1 = t_0 + \Delta t$

How do we get a new population value based on the starting population (i.e. $P(t_1)$)?

$$P(t_1) = P(t_0) + (\frac{dP}{dt} * \Delta t)$$

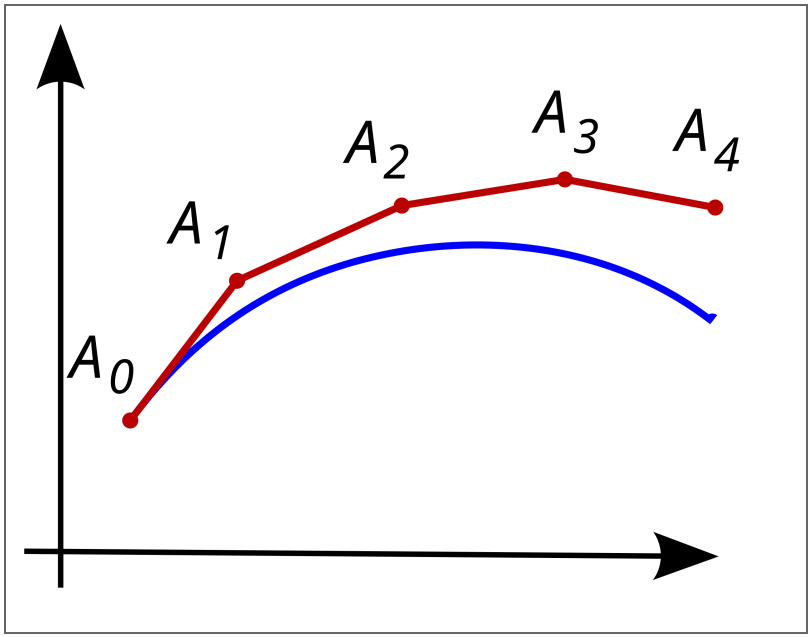
This is our **update equation**. For the next population value we want, $P(t_2)$, it would be:

$$P(t_2) = P(t_1) + (\frac{dP}{dt} * \Delta t)$$

Sometimes we'll simplify these to look like:

$$P_2 = P_1 + (\frac{dP}{dt} * \Delta t)$$

In a generic sense, visually, this might look something like:



Note: this visual is not for our population model

So if I know I'm going to need to calculate this "update" many times, it makes sense to put the derivative into a function

Remember, this is the the function I can use to update my population:

$$\frac{dP}{dt} = kP\left(1 - \frac{P}{C}\right)$$

So, I can turn it into a function that looks like:

In [1]:

```
# Define a function that takes the current population
# and returns the value of the derivative at that point
def derivs(p_current, k, C):
    dpdt = k*p_current*(1 - (p_current/C))
    return dpdt
```

I should quickly test my function

In [2]:

```
# Put some testing code here
print(derivs(1e9,0.02,12e9))
```

```
18333333.333333332
```

Then I can do something where I start with my initial value, and march it forward in time using a loop (either a `for` loop or a `while` loop)
My plan should look something like:

1. Define a starting population value
2. To start, make my "current" population value the starting population value
3. For a set number of time steps:
    - Multiply the result of my derivative function by my chosen time step size and add that to my current population value to calculate the new population.
    - Make sure that in each iteration, the new value becomes the current value before the next time step.
4. Along the way, make sure I'm storing my population values.

Goals for today

- Use a function for the derivative of the population to calculate an array of population values over a range of time.
- Compare this numerical solution to the exact solution, using our equation from previous assignments.
- Explore how changing the time-step changes the numerical solution.

Looking forward - Day 9 PCA: Trying to use ODEs to evolve a system for physical motion using both update equations and `solve_ivp`

- Test out your ODEs solutions for another system
- Explore how we can use a pre-built tool, called `solve_ivp` to do this.